AI-Based Personalized Course Recommendation and Learning Path System Using A* Search

Shobbika T
Department of Computer Science and Engineering
240701502
shobbika.t.2024.cse@rajalakshmi.edu.in

Sona sree R P
Department of Computer Science and Engineering
240701521
sonasree.rp.2024.cse@rajalakshmi.edu.in

Rajalakshmi Engineering College (verify official institution name before submission)

Abstract— Learners often face difficulty selecting courses in an order that matches their existing knowledge, career objectives, and available study time. This paper presents an AI-based personalized course recommendation and learning path system that uses the A* search algorithm to identify a sequence of courses from a learner's current skill state toward a target career goal. The system represents course prerequisites and skill coverage as a search space, accumulates course-related costs, and uses a heuristic estimate to guide exploration. A learner profile captures knowledge level, acquired skills, career goal, interests, and weekly study availability. The resulting path is presented as an ordered learning plan with cost information and can be updated as courses are completed. The paper describes the system architecture, state representation, search procedure, and evaluation approach. Since verified benchmark measurements were not supplied, the evaluation section defines metrics and a comparison method rather than claiming experimental results. The approach demonstrates how heuristic search can support explainable, goal-oriented learning recommendations.

Keywords— A* search, personalized learning, course recommendation, heuristic search, learning path, prerequisite planning.

I. INTRODUCTION

Online learning platforms provide a large number of courses, but learners may not know which courses to take first or how to connect individual courses to a career goal. A course that is useful for one learner may be redundant for another because their prior skills, time availability, and target roles differ. A personalized learning path can address this problem by selecting relevant courses and arranging them in a prerequisite-aware sequence.

This project models learning-path generation as a search problem. The learner's current knowledge forms the initial state, while the target career skills define the goal condition. A* search ranks candidate states using an accumulated path cost and a heuristic estimate of the remaining cost. The interface presents the resulting course sequence and supports progress updates when a learner completes a course.

II. RELATED WORK

Hart, Nilsson, and Raphael introduced a formal basis for heuristic determination of minimum-cost paths and established the A* search approach [1]. A* evaluates candidate paths using $f(n) = g(n) + h(n)$, where $g(n)$ is the cost already incurred and $h(n)$ estimates the remaining cost to a goal. Russell and Norvig provide a broader treatment of informed search and its properties in artificial intelligence [2].

Recommender-system research studies methods for helping users identify relevant items from large collections. Ricci, Rokach, and Shapira discuss recommender-system foundations and challenges, including personalization and evaluation [3]. The proposed project combines a course knowledge base with explicit goal-directed search. It should therefore be understood as a search-based recommender, not as a trained machine-learning model unless a separate learned component is implemented.

III. PROBLEM FORMULATION

Let the initial learner state contain the courses and skills already acquired. Each course is represented by attributes such as its title, duration, difficulty, prerequisites, and skills covered. A course is eligible for expansion only when its prerequisite conditions are satisfied. The goal test checks whether the learner's acquired or planned skills cover the skill requirements associated with the selected career goal.

The path cost is accumulated over the courses selected. In the application's displayed cost model, a course cost is based on duration hours plus a difficulty penalty (Beginner = 0, Intermediate = 5, Advanced = 10). These penalty values are search weights, not additional study hours or monetary costs. The cost model should be described consistently with the implementation used in the submitted project.

IV. SYSTEM ARCHITECTURE

The system can be organized into five main modules: (1) Student Profile and Input Module, which collects the learner's knowledge level, acquired skills, career goal, interests, and weekly study hours; (2) Course Knowledge Base and Prerequisite Module, which stores course metadata and prerequisite relationships; (3) A* Search and Recommendation Module, which generates candidate learning paths and evaluates them using g(n), h(n), and f(n); (4) Result Presentation Module, which displays the recommended sequence, estimated cost, and study plan; and (5) Error Handling and Validation, which checks inputs and handles cases where no valid path is found.

The progress tracker updates the learner's current state after course completion and can trigger a new search. A study-planning component maps course durations to available study days and weekly hours. The architecture is modular so that the course catalogue, cost model, heuristic, and presentation layer can be maintained independently.

V. A* SEARCH METHODOLOGY

A* uses the evaluation function $f(n) = g(n) + h(n)$. Here, $g(n)$ is the accumulated cost from the initial state to the current state, and $h(n)$ estimates the remaining cost to satisfy the career goal. The search repeatedly selects a frontier state with the lowest f-value, checks the goal condition, and expands eligible courses whose prerequisites are satisfied. A successor state adds the skills or course completion associated with the selected course and updates the accumulated cost.

The recommendation process is: (1) validate the learner profile and target goal; (2) construct the initial state from known skills and completed courses; (3) identify eligible courses; (4) calculate g, h, and f for candidate states; (5) continue expanding the lowest-ranked state until the goal is reached or the frontier is exhausted; and (6) return the selected course sequence and summary statistics. The quality and optimality of A* depend on the state representation, edge costs, and heuristic. Claims of optimality require a suitable heuristic and non-negative costs.

VI. IMPLEMENTATION AND USER WORKFLOW

The application workflow begins with student profile entry. The learner selects a career goal, indicates acquired skills and knowledge level, and specifies weekly study availability. The system then generates a personalized path and displays the course order alongside cost-related information. The learner can review the A* explanation, track completed courses, and create a study schedule based on available days and hours.

The course catalogue and dataset-management functions support maintaining the knowledge base. An algorithm-evaluation component can compare A* with Uniform-Cost Search (UCS) using the same graph, start state, goal test, and edge costs. Implementation details in this paper describe the intended system design; exact libraries, data structures, and interface technologies should be checked against the final source code before submission.

VII. EVALUATION PLAN

A reproducible evaluation should record the number of expanded states, total path cost, number of courses recommended, runtime, and whether the goal requirements are satisfied. A* and UCS should be run on identical course data and learner profiles. Tests should include a learner with no prior skills, a learner with partial prerequisites, a learner who has already completed several courses, and a goal for which no valid path exists.

No verified execution logs or benchmark dataset were provided for this paper; consequently, no numerical performance results are asserted. Once measurements are collected, report the test environment, dataset size, repeated-run procedure, and actual results. Any screenshots used in the final report should be labelled as illustrative application outputs unless they were produced during a documented experiment.

VIII. LIMITATIONS AND FUTURE WORK

Recommendation quality depends on the accuracy and completeness of course metadata, prerequisites, and career-skill mappings. A heuristic that is poorly designed may guide search inefficiently; if it overestimates the remaining cost, the usual optimality guarantee may not hold. The cost model also simplifies course difficulty and does not directly measure learning outcomes, learner preferences, or course quality.

Future work may include validating the heuristic against known optimal paths, evaluating on a larger curated course catalogue, collecting learner feedback, incorporating course ratings and learning outcomes, and adapting the study plan as availability changes. Any machine-learning-based personalization should be separately specified and evaluated rather than implied by the use of the term "AI."

IX. CONCLUSION

This paper presented a search-based personalized course recommendation and learning-path system using A*. By representing learner knowledge, course prerequisites, and career requirements as a goal-directed search problem, the system can generate an ordered course sequence and explain its accumulated and estimated costs. A progress tracker and study planner can make the recommendation actionable over time. Future evaluation should use reproducible experiments and verified measurements to assess path quality, search efficiency, and practical usefulness.

REFERENCES

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968, doi: 10.1109/TSSC.1968.300136.

[2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] F. Ricci, L. Rokach, and B. Shapira, “Recommender Systems: Introduction and Challenges,” in Recommender Systems Handbook, 3rd ed. Cham, Switzerland: Springer, 2022, doi: 10.1007/978-1-4899-7637-6_1.

[4] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/